MIABE HACKATHON 2026 — NIGER

PharmaTrace

Livrable Phase 3 (Finale)

Équipe : TrustGrid

Date : Juin 2026

1. Introduction & Présentation Générale

PharmaTrace est une solution technologique décentralisée conçue pour lutter contre le fléau des médicaments contrefaits ou falsifiés au Niger. En s'appuyant sur une architecture de type **Blockchain-lite** et des QR codes immuables, PharmaTrace sécurise la chaîne logistique médicale nigérienne de bout en bout (fabrication/importation → transfert logistique → pharmacie → patient final).

Ce document détaille l'architecture logicielle, le schéma cryptographique immuable, la structure de la base de données PostgreSQL, les politiques de sécurité active (RLS), les mécanismes de détection automatique de fraudes, ainsi que le plan de déploiement technique de la plateforme.

2. Architecture Technique Globale

La solution est construite sur une architecture moderne, découplée et hautement disponible, favorisant la rapidité d'exécution sur le terrain (temps de scan < 3 secondes) même en conditions de réseau dégradé.

Composant	Technologie choisie	Rôle principal
Application Mobile / Web	React Native, Expo (SDK 54), TypeScript	Interface unique pour tous les acteurs (Android/iOS et Web). Accès caméra pour scan rapide.
Moteur de Style	Tailwind CSS / NativeWind (v4)	Charte graphique moderne, ultra-rapide, responsive et premium (couleurs du drapeau nigérien revisitées).
Base de Données & Backend	Supabase, PostgreSQL (v15)	Hébergement du registre, gestion de l'authentification des acteurs agrées, exécution des triggers de sécurité.
Intégrité Blockchain-lite	Génération de Hash SHA-256 + Triggers de non-répudiation	Garantie d'immuabilité et de traçabilité sans le coût environnemental ni la lenteur d'une blockchain publique.
Hébergement Web	Vercel (App Web & Site Vitrine)	Déploiement continu et distribution mondiale avec un temps de réponse minimal.

3. Système Blockchain-Lite & Cryptographie

Pour garantir l'intégrité absolue sans l'empreinte carbone et la complexité des transactions blockchain traditionnelles PharmaTrace utilise un système de **blockchain-lite cryptographique**.

Génération du Hash d'Identité Unique

À chaque fois qu'un fabricant agréé enregistre un lot, le système calcule un hash SHA-256 unique et immuable représentant la signature numérique de ce lot physique.

> **Formule de calcul du Hash SHA-256 :**
>
> `hash_sha256 = SHA256(numero_lot || nom_medicament || date_fabrication || fabricant_id || created_at)`

Ce hash est stocké dans le registre de manière immuable (protégé par des règles de base de données interdisant toute modification après insertion) et encodé dans le QR code apposé sur les emballages.

Mécanisme de Détection de Fraude (Alerte Automatique)

Lorsqu'un utilisateur (patient ou pharmacien) scanne un médicament, le système envoie le hash extrait du QR code à Supabase. Deux cas se présentent :

1. **Le Hash existe :** Le système retourne l'état d'authenticité (Authentique, En transit, Périmé, etc.).
2. **Le Hash n'existe pas :** Un trigger PostgreSQL s'exécute instantanément, enregistrant une **alerte critique** ("lot_inconnu") localisée géographiquement (ville du scan) pour notifier la Direction de la Pharmacie (DPH).

4. Schéma de la Base de Données

La base de données PostgreSQL est structurée de manière hautement relationnelle avec des index optimisés pour le volume.

4.1 Tables principales

Table : `utilisateurs`

Stocke les informations d'identité des acteurs agréés au Niger.

```
CREATE TABLE public.utilisateurs (
  id               UUID        PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email            VARCHAR(255) NOT NULL,
  nom              VARCHAR(100) NOT NULL,
  nom_entite       VARCHAR(255) NOT NULL, -- Ex: "Pharmacie du Centre Niamey"
  role             user_role   NOT NULL DEFAULT 'fabricant', -- fabricant, pharmacie, agen
  telephone        VARCHAR(20),
  adresse          TEXT,
  ville            VARCHAR(100),
  numero_agrement  VARCHAR(100), -- Numéro d'agrément officiel nigérien
  statut           VARCHAR(10)  NOT NULL DEFAULT 'actif',
  created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Table : `lots_medicaments`

Enregistre chaque lot de médicament authentique fabriqué ou importé.

```
CREATE TABLE public.lots_medicaments (
  id                  UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
  numero_lot          VARCHAR(100) UNIQUE NOT NULL,
  nom_medicament      VARCHAR(255) NOT NULL,
  dosage              VARCHAR(100),
  quantite_initiale   INTEGER      NOT NULL CHECK (quantite_initiale > 0),
  quantite_actuelle   INTEGER      NOT NULL,
  date_fabrication    DATE         NOT NULL,
  date_expiration     DATE         NOT NULL,
  fabricant_id        UUID         NOT NULL REFERENCES public.utilisateurs(id),
  hash_sha256         VARCHAR(64)  UNIQUE NOT NULL,
  qr_code_url         TEXT,
  statut_actuel       statut_lot   NOT NULL DEFAULT 'en_stock_fabricant'
);
```

Table : `transferts`

Gère les flux logistiques entre fabricants et pharmacies.

```
CREATE TABLE public.transferts (
  id                  UUID             PRIMARY KEY DEFAULT gen_random_uuid(),
  lot_id              UUID             NOT NULL REFERENCES public.lots_medicaments(id),
  expediteur_id       UUID             NOT NULL REFERENCES public.utilisateurs(id),
  destinataire_id     UUID             NOT NULL REFERENCES public.utilisateurs(id),
  quantite_transferee INTEGER          NOT NULL CHECK (quantite_transferee > 0),
  date_reception      TIMESTAMPTZ,
  statut              statut_transfert NOT NULL DEFAULT 'en_attente' -- en_attente, recept
);
```

Table : `historique_scans`

tous les scans de vérification pour alimenter les statistiques de la DPH.

```
CREATE TABLE public.historique_scans (
  id                 UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
  hash_scanne        VARCHAR(64)  NOT NULL,
  lot_id             UUID         REFERENCES public.lots_medicaments(id),
  type_scanneur      VARCHAR(20)  NOT NULL, -- patient, pharmacien, agent_dph
  resultat           resultat_scan NOT NULL, -- authentique, suspect, non_trouve, expire
  localisation_ville VARCHAR(100), -- Ex: "Zinder", "Niamey"
  created_at         TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);
```

Table : `alertes`

Stocke les alertes en cas d'activités suspectes détectées.

```
CREATE TABLE public.alertes (
  id              UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  lot_id          UUID          REFERENCES public.lots_medicaments(id),
  scan_id         UUID          REFERENCES public.historique_scans(id),
  type_alerte     VARCHAR(50)   NOT NULL, -- lot_inconnu, qr_duplique, lot_expire_en_circu
  description     TEXT          NOT NULL,
  niveau          niveau_alerte NOT NULL, -- info, avertissement, critique
  statut          statut_alerte NOT NULL DEFAULT 'ouverte', -- ouverte, en_cours_traitemen
  ville_concernee VARCHAR(100),
  created_at      TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);
```

5. Politiques de Sécurité RLS (Row Level Security)

PharmaTrace applique des politiques de sécurité strictes au niveau de PostgreSQL (RLS) pour interdire tout accès non autorisé aux données sensibles, tout en laissant les informations de vérification ouvertes au public.

- **Sécurité des Lots :**
  - `Lecture publique :` Tout le monde (y compris les patients non connectés) peut lire les lots en filtrant par hash pour vérifier l'authenticité (`USING (true)`).
  - `Création & Modification :` Restreinte uniquement au fabricant propriétaire du lot, vérifiée via son ID utilisateur et son rôle (`fabricant_id = auth.uid() AND get_my_role() = 'fabricant'`).

- **Sécurité des Transferts :**
  - Visibilité réservée à l'expéditeur (fabricant), au destinataire (pharmacie) et aux agents officiels de la DPH pour contrôle.

- **Sécurité des Alertes :**
  - Les alertes sont générées automatiquement par le système lors d'un scan suspect. Seul un agent de la DPH authentifié (`get_my_role() = 'agent_dph'`) peut les lire, les modifier ou marquer une enquête comme résolue.

6. Déploiement & Maintenance

La solution est hébergée et déployée selon les standards modernes du Cloud de manière à garantir un fonctionnement permanent sans interruption de service.

Hébergement

- **Application Web & Site Vitrine :** Déployés sur **Vercel** avec mise en cache des assets statiques.
- **Base de données & Authentification :** Hébergées sur l'infrastructure Cloud de **Supabase**, avec des réplicas de bases de données pour assurer la résilience.

Procédure de mise en production

```
# 1. Installer les dépendances du projet
npm install

# 2. Lancer les migrations de base de données Supabase
npx supabase db push

# 3. Compiler le build web de production
npm run build

# 4. Compiler l'APK Android de production avec EAS Build
npx eas build --platform android --profile production
```